



المدرسة العليا
للتكنولوجيا - قلعة السراغنة
ÉCOLE SUPÉRIEURE DE TECHNOLOGIE
D'EL KELÂA DES SRAGHNA

**Université Cadi Ayyad
École Supérieure de Technologie (EST)
El Kelâa des Sraghna**

Filière DUT : Ingénierie des Données

Rapport de Projet de Fin d'Etudes

Développement d'un Agent Conversationnel Intelligent pour l'Analyse des
Données d'Entreprise

Présenté et soutenue publiquement par :

**Mr. Yassine Ennajai
Mr. Mortaja Elmehdi
Mr. Elhimri Hamza**

Le 1 Juillet 2025

Membre de jury

Pr.

Pr.

EST, El Kelâa des
Sraghna

EST, El Kelâa des
Sraghna

**Président et
Examineur
Encadrante**

Remerciements

Je tiens à exprimer ma profonde gratitude et mes sincères remerciements à mon encadrant **SOUKAINA ETTREFAOUI**, pour son accompagnement tout au long de la réalisation de ce projet.

Je lui suis particulièrement reconnaissant pour sa disponibilité, ses conseils pertinents, ainsi que pour l'attention qu'il a portée à l'évolution de ce travail. Ses orientations, sa rigueur scientifique et son soutien constant ont largement contribué à l'aboutissement de ce projet.

Je lui adresse ainsi ma sincère reconnaissance pour son encadrement de qualité et pour l'intérêt qu'il a manifesté envers ce travail.

Contents

Introduction Générale	1
1. Contexte et motivation du projet	1
2. Problématique	1
3. Objectifs du projet.....	2
4. Méthodologie générale	2
5. Organisation du rapport	3
Chapitre 1 : État de l'art et concepts fondamentaux	3
1.1 Intelligence artificielle et analyse de données	3
1.1.1 Définition et périmètre de l'intelligence artificielle.....	3
1.1.2 Évolution historique de l'intelligence artificielle	4
1.1.3 Les modèles de langage (LLMs).....	4
1.1.4 Application à l'analyse de données	5
1.2 Business Intelligence et analyse décisionnelle	5
1.2.1 Définition et objectifs	5
1.2.2 Composants d'un système de Business Intelligence	6
1.2.3 Limites des outils traditionnels.....	6
1.2.4 Vers une Business Intelligence augmentée	6
Chapitre 2 : Conception et architecture du système	7
2.1 Architecture globale du système	7
2.1.1 Principes architecturaux.....	7
2.1.2 Vue d'ensemble de l'architecture.....	7
Diagramme d'architecture du système	8
2.1.3 Flux de traitement d'une requête	9
2.1.4 Double mode de déploiement	9
2.2 Description détaillée des composants	9
2.2.1 L'orchestrateur (AgentWorkflow).....	9
2.2.2 L'agent d'analyse (Analytics Agent)	10
2.2.3 L'agent RAG (Retrieval Augmented Generation)	10
2.2.4 Le générateur de rapports (Report Generator).....	11
2.2.5 Gestion des sessions et de la mémoire.....	12
2.3 Modélisation des données.....	12
2.3.1 Structure du dataset.....	12
2.3.2 Métadonnées	12
Chapitre 3 : Développement et validation du système	13
3.1 Environnement de développement.....	13
3.1.1 Configuration technique.....	13
3.1.2 Structure du projet.....	13
3.2 Développement des composants principaux.....	14

3.2.1 Orchestrateur (AgentWorkflow)	14
3.2.2 Analytics Agent.....	15
3.2.3 RAG Agent	16
3.2.4 Report Generator	17
3.2.5 Gestion des sessions et contexte	18
3.3 Modélisation des données	19
3.3.1 Structure du dataset	19
3.3.2 Métadonnées exposées	20
3.4 Flux de traitement d'une requête.....	20
3.5 Exemple d'interaction complète	20
3.6 Validation et résultats.....	22
3.6.1 Tests réalisés	22
3.6.2 Résultats obtenus.....	22
3.6.3 Retours utilisateurs.....	22
3.7 Perspectives d'amélioration	22
Conclusion générale.....	23

Introduction Générale

1. Contexte et motivation du projet

Avec l'augmentation massive des données dans les entreprises, il devient essentiel de disposer d'outils capables d'analyser rapidement ces informations afin d'aider à la prise de décision. Les entreprises collectent quotidiennement de grandes quantités de données provenant de différentes sources telles que les ventes, les transactions et les interactions avec les clients.

Selon plusieurs études récentes, le volume mondial de données connaît une croissance exponentielle. Les organisations produisent et stockent des volumes considérables d'informations chaque jour. Cette explosion de données représente une opportunité importante pour améliorer la prise de décision stratégique.

Cependant, malgré cette abondance de données, les entreprises rencontrent encore des difficultés pour transformer ces informations en connaissances exploitables. Une grande partie des décisions organisationnelles ne repose pas encore entièrement sur des analyses de données structurées.

L'une des principales raisons de cette situation est la complexité des outils analytiques existants. La plupart des plateformes de Business Intelligence nécessitent des compétences techniques avancées, notamment en programmation, en manipulation de bases de données ou en analyse statistique.

Dans ce contexte, l'intelligence artificielle et les modèles de langage offrent de nouvelles perspectives. Ces technologies permettent aujourd'hui de concevoir des systèmes capables de comprendre des questions formulées en langage naturel et de générer automatiquement des analyses pertinentes.

Ainsi, ce projet vise à développer une application intelligente permettant aux utilisateurs d'interroger leurs données de ventes à l'aide de questions simples et d'obtenir instantanément des analyses, des visualisations et des rapports.

2. Problématique

Malgré l'existence de nombreux outils d'analyse de données, plusieurs limitations persistent dans les environnements professionnels.

Premièrement, la complexité technique constitue une barrière importante pour les utilisateurs non spécialisés. L'utilisation d'outils d'analyse traditionnels nécessite souvent la maîtrise de langages comme SQL, Python ou R.

Deuxièmement, le délai d'accès à l'information peut être relativement long. Dans de nombreuses organisations, les demandes d'analyse passent par des équipes spécialisées, ce qui ralentit considérablement le processus décisionnel.

Troisièmement, il existe souvent une fracture entre les équipes métiers et les spécialistes de la data. Les utilisateurs métiers ont parfois des difficultés à exprimer précisément leurs besoins analytiques, tandis que les data analysts peuvent manquer de contexte métier.

Quatrièmement, certaines solutions analytiques professionnelles peuvent représenter un investissement financier important, en particulier pour les petites et moyennes entreprises.

Enfin, la question de la confidentialité des données est devenue un enjeu majeur. De nombreuses solutions basées sur l'intelligence artificielle nécessitent l'envoi des données vers des services externes, ce qui peut poser des problèmes de sécurité et de conformité.

Dans ce contexte, la problématique principale de ce projet peut être formulée comme suit :

Comment concevoir un assistant intelligent capable de comprendre les questions des utilisateurs en langage naturel, d'analyser automatiquement les données d'entreprise, de générer des visualisations pertinentes et de produire des rapports synthétiques, tout en garantissant simplicité d'utilisation, performance et sécurité des données ?

3. Objectifs du projet

L'objectif principal de ce projet est de développer une application intelligente permettant l'analyse automatisée des données d'entreprise à partir de questions formulées en langage naturel.

L'objectif général consiste à concevoir un système interactif permettant aux utilisateurs d'interroger leurs données sans avoir besoin de compétences techniques avancées.

Plusieurs objectifs spécifiques ont été définis pour atteindre ce but.

Le premier objectif est de développer un module capable de comprendre les questions des utilisateurs et d'interpréter leurs intentions.

Le deuxième objectif est de concevoir un agent d'analyse capable d'exécuter automatiquement des calculs statistiques à partir des données disponibles.

Le troisième objectif consiste à développer un module de génération de rapports permettant de produire des documents synthétiques au format PDF.

Le quatrième objectif est de concevoir un orchestrateur intelligent chargé d'analyser les requêtes et de les orienter vers le composant approprié du système.

Enfin, un dernier objectif consiste à développer une interface utilisateur simple et intuitive facilitant l'interaction entre l'utilisateur et le système.

4. Méthodologie générale

La réalisation de ce projet s'appuie sur une méthodologie structurée en plusieurs phases.

La première phase concerne l'analyse des besoins et la définition des spécifications fonctionnelles et techniques du système.

La deuxième phase est consacrée à la conception de l'architecture globale de la solution. Cette étape inclut la modélisation des composants du système et la définition de leurs interactions.

La troisième phase correspond au développement de l'application. Cette étape comprend l'implémentation des différents modules du système, notamment l'orchestrateur, l'agent d'analyse et le générateur de rapports.

La quatrième phase concerne les tests et la validation du système. Différents scénarios d'utilisation sont testés afin d'évaluer les performances et la fiabilité de l'application.

La dernière phase consiste à documenter le projet et à préparer le déploiement de l'application.

5. Organisation du rapport

Ce rapport est structuré en trois chapitres principaux.

Le premier chapitre présente les concepts fondamentaux liés à l'intelligence artificielle, à l'analyse de données et aux technologies utilisées dans ce projet.

Le deuxième chapitre décrit l'analyse des besoins et la conception de la solution proposée. L'architecture globale du système et les différents composants sont détaillés.

Le troisième chapitre présente l'implémentation de l'application ainsi que les résultats obtenus lors des tests réalisés.

Une conclusion générale permet enfin de synthétiser les contributions du projet et de proposer des perspectives d'amélioration.

Chapitre 1 : État de l'art et concepts fondamentaux

1.1 Intelligence artificielle et analyse de données

1.1.1 Définition et périmètre de l'intelligence artificielle

L'intelligence artificielle (IA) constitue aujourd'hui un domaine majeur de l'informatique et des sciences cognitives. Elle vise à concevoir des systèmes capables d'effectuer des tâches nécessitant habituellement l'intelligence humaine, telles que la reconnaissance visuelle, la compréhension du langage naturel, la prise de décision ou encore l'apprentissage automatique.

Le terme « intelligence artificielle » a été introduit en 1956 par John McCarthy lors de la conférence de Dartmouth, considérée comme le point de départ officiel de cette discipline.

Depuis cette période, l'IA a connu une évolution considérable, passant des approches symboliques basées sur des règles logiques à des méthodes statistiques et d'apprentissage automatique beaucoup plus avancées.

Aujourd'hui, l'intelligence artificielle est utilisée dans de nombreux domaines tels que la médecine, la finance, l'industrie, le commerce électronique ou encore l'analyse de données. Son objectif principal est d'améliorer la capacité des systèmes informatiques à traiter l'information, apprendre à partir des données et assister l'être humain dans la prise de décision.

1.1.2 Évolution historique de l'intelligence artificielle

L'évolution de l'intelligence artificielle peut être divisée en plusieurs grandes périodes qui ont marqué son développement.

La première période correspond aux débuts de l'IA entre les années 1950 et 1970. Durant cette phase, les chercheurs se concentraient principalement sur l'intelligence artificielle symbolique. Les systèmes développés étaient basés sur des règles logiques et des connaissances explicites fournies par des experts humains. Ces systèmes, appelés systèmes experts, étaient capables de résoudre des problèmes dans des domaines spécifiques, mais restaient limités face à des situations complexes ou imprévues.

La seconde période correspond à l'essor du machine learning entre les années 1980 et 2010. Durant cette phase, les chercheurs ont commencé à développer des algorithmes capables d'apprendre directement à partir des données. Cette approche a permis d'améliorer considérablement les performances dans des tâches telles que la classification, la régression ou le clustering. Des algorithmes comme les réseaux de neurones artificiels, les machines à vecteurs de support (SVM) ou les forêts aléatoires sont devenus très populaires.

La troisième période correspond à la révolution du deep learning entre 2010 et 2020. Grâce à l'augmentation de la puissance de calcul et à la disponibilité de grandes quantités de données, les réseaux de neurones profonds ont permis des avancées majeures dans plusieurs domaines comme la vision par ordinateur, la reconnaissance vocale et le traitement automatique du langage naturel.

Enfin, la période actuelle est marquée par l'émergence des grands modèles de langage, appelés Large Language Models (LLMs). Ces modèles sont capables de comprendre et de générer du texte avec un niveau de précision très élevé, ouvrant ainsi de nouvelles perspectives dans le domaine des applications intelligentes.

1.1.3 Les modèles de langage (LLMs)

Les modèles de langage de grande taille représentent une avancée majeure dans le domaine du traitement automatique du langage naturel. Ils sont conçus pour analyser, comprendre et générer du texte en se basant sur de très grandes quantités de données textuelles.

Le fonctionnement de ces modèles repose principalement sur l'architecture Transformer introduite en 2017 par Vaswani et ses collaborateurs. Cette architecture utilise un mécanisme

d'attention permettant d'analyser les relations entre les mots d'une phrase de manière plus efficace que les approches traditionnelles basées sur les réseaux récurrents.

Les modèles de langage passent généralement par deux étapes principales. La première étape est le pré-entraînement, durant laquelle le modèle est entraîné sur d'immenses corpus de textes provenant de livres, d'articles scientifiques ou de pages web. Cette phase permet au modèle d'acquérir une compréhension générale du langage. La seconde étape est appelée fine-tuning et consiste à adapter le modèle à des tâches spécifiques à l'aide de données plus ciblées.

Grâce à cette approche, les LLMs sont capables de réaliser de nombreuses tâches telles que la traduction automatique, la génération de texte, la rédaction de code informatique, la synthèse de documents ou encore la réponse à des questions complexes.

1.1.4 Application à l'analyse de données

L'intégration des modèles de langage dans le domaine de l'analyse de données ouvre de nouvelles perspectives pour le développement d'applications intelligentes.

Tout d'abord, ces modèles permettent d'interpréter des questions formulées en langage naturel par les utilisateurs. Par exemple, un utilisateur peut poser une question comme « quel mois a généré le plus de ventes ? » sans avoir besoin d'écrire une requête SQL ou du code Python.

Ensuite, les modèles de langage peuvent générer automatiquement du code d'analyse de données en utilisant des bibliothèques comme Pandas. Cela permet d'automatiser certaines étapes de traitement et d'accélérer considérablement le processus d'analyse.

Les LLMs peuvent également expliquer les résultats obtenus en produisant des descriptions claires et compréhensibles en langage naturel. Cette capacité facilite l'interprétation des analyses par des utilisateurs non techniques.

Enfin, ces modèles peuvent tenir compte du contexte d'une conversation afin d'offrir une interaction plus naturelle et plus intuitive entre l'utilisateur et le système d'analyse.

1.2 Business Intelligence et analyse décisionnelle

1.2.1 Définition et objectifs

La Business Intelligence, souvent abrégée BI, désigne l'ensemble des méthodes, outils et technologies permettant de collecter, stocker, analyser et présenter les données d'une organisation dans le but d'améliorer la prise de décision.

Les systèmes de Business Intelligence permettent aux entreprises de transformer les données brutes en informations exploitables. Ces informations aident les décideurs à comprendre les performances de l'organisation, à identifier les tendances du marché et à anticiper les évolutions futures.

Les objectifs principaux de la Business Intelligence sont notamment l'amélioration de la prise de décision, l'optimisation des processus opérationnels, la détection de nouvelles opportunités commerciales et le suivi des indicateurs de performance.

1.2.2 Composants d'un système de Business Intelligence

Un système complet de Business Intelligence repose généralement sur plusieurs composants fondamentaux.

Le premier composant concerne la collecte et l'intégration des données. Cette étape est souvent réalisée grâce à des processus ETL (Extract, Transform, Load) qui permettent d'extraire les données à partir de différentes sources, de les transformer afin de les rendre exploitables, puis de les charger dans un entrepôt de données centralisé.

Le second composant concerne le stockage et la modélisation des données. Les données sont organisées dans des structures adaptées à l'analyse, comme les schémas en étoile ou en flocon, afin de faciliter les requêtes analytiques.

Le troisième composant concerne l'analyse et le reporting. Des outils spécialisés permettent aux utilisateurs d'explorer les données, de créer des rapports et de suivre différents indicateurs de performance.

Enfin, la visualisation des données joue un rôle essentiel. Les graphiques, tableaux de bord et visualisations interactives permettent de présenter les résultats de manière claire et compréhensible.

1.2.3 Limites des outils traditionnels

Malgré leurs nombreux avantages, les outils traditionnels de Business Intelligence présentent certaines limites.

Tout d'abord, leur utilisation nécessite souvent des compétences techniques spécifiques. Les utilisateurs doivent généralement maîtriser des outils complexes ou des langages comme SQL afin d'interroger les bases de données.

Ensuite, la création de rapports ou de tableaux de bord peut nécessiter beaucoup de temps, surtout lorsque les données proviennent de différentes sources.

De plus, certaines solutions professionnelles impliquent des coûts de licence relativement élevés, ce qui peut constituer un obstacle pour certaines organisations.

Enfin, l'accès à l'information n'est pas toujours immédiat, car il dépend souvent de l'intervention d'analystes ou d'équipes techniques spécialisées.

1.2.4 Vers une Business Intelligence augmentée

Face à ces limitations, de nouvelles approches basées sur l'intelligence artificielle ont émergé. On parle alors de Business Intelligence augmentée.

La BI augmentée vise à intégrer des technologies d'intelligence artificielle afin d'automatiser certaines tâches liées à l'analyse de données. Cela inclut notamment l'automatisation de la préparation des données, la génération automatique de visualisations et la détection d'insights pertinents.

L'un des principaux avantages de cette approche est la possibilité d'interroger les données en langage naturel. Les utilisateurs peuvent ainsi poser des questions directement en français ou dans une autre langue, sans avoir besoin de connaissances techniques avancées.

Cette évolution permet de démocratiser l'accès à l'analyse de données et de rendre les outils décisionnels plus accessibles à un plus grand nombre d'utilisateurs.

Chapitre 2 : Conception et architecture du système

2.1 Architecture globale du système

2.1.1 Principes architecturaux

L'architecture du système repose sur une approche modulaire permettant de séparer les différentes responsabilités de l'application. Cette conception facilite la maintenance du système et permet d'ajouter de nouvelles fonctionnalités sans modifier l'ensemble de l'architecture.

Le premier principe fondamental est la **modularité**. Chaque fonctionnalité principale est encapsulée dans un module indépendant appelé agent. Chaque agent possède un rôle spécifique dans le traitement des requêtes.

Le second principe est l'**orchestration centralisée**. Un composant central appelé orchestrateur reçoit toutes les requêtes et décide du traitement approprié. Cela permet de coordonner efficacement les différents modules.

Un autre principe important est la **séparation des responsabilités**. Les traitements liés à l'analyse de données, à la génération de réponses intelligentes et à la création de rapports sont gérés par des composants distincts.

Enfin, l'architecture a été conçue pour être **extensible**. Il est possible d'ajouter facilement de nouveaux agents ou de nouvelles fonctionnalités sans modifier la structure globale du système.

2.1.2 Vue d'ensemble de l'architecture

L'architecture du système repose sur plusieurs composants qui interagissent afin de traiter les requêtes des utilisateurs.

L'utilisateur accède au système via une interface web développée avec le framework Flask. Cette interface permet de poser des questions en langage naturel, d'afficher les résultats des analyses et de générer des rapports.

Les requêtes sont ensuite envoyées à un composant central appelé **orchestrateur (AgentWorkflow)**. Celui-ci analyse les requêtes et décide quel agent doit être utilisé pour produire la réponse.

Trois agents principaux composent le système :

- **Analytics Agent** : chargé d'analyser les données.
- **RAG Agent** : responsable des réponses basées sur les connaissances.
- **Report Generator** : responsable de la génération des rapports PDF.

Chaque agent utilise des outils spécifiques pour accomplir ses tâches.

Diagramme d'architecture du système

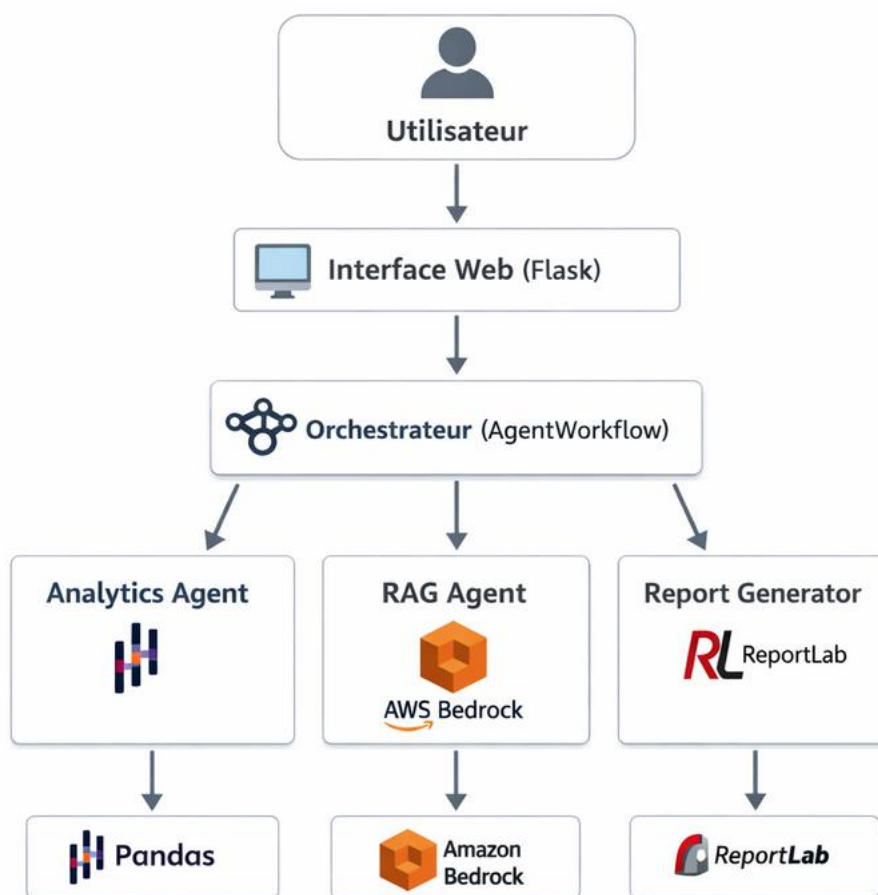


Figure1 :Diagramme d'architecture du système 1

Description du diagramme :

- L'utilisateur interagit avec l'application via l'interface web.
- L'interface envoie la requête à l'orchestrateur.
- L'orchestrateur analyse la requête et la dirige vers l'agent approprié.
- Chaque agent utilise un outil spécifique pour traiter la demande.

2.1.3 Flux de traitement d'une requête

Le traitement d'une requête utilisateur suit plusieurs étapes.

Dans un premier temps, l'utilisateur saisit une question dans l'interface web. Cette requête est ensuite envoyée au serveur qui la transmet à l'orchestrateur.

L'orchestrateur analyse la requête afin d'identifier son intention. Cette étape inclut la détection de la langue utilisée ainsi que la classification du type de question.

Une fois l'analyse terminée, l'orchestrateur redirige la requête vers l'agent spécialisé correspondant. L'agent effectue alors le traitement nécessaire et renvoie les résultats à l'orchestrateur.

Enfin, la réponse est formatée et renvoyée à l'interface utilisateur pour être affichée.

2.1.4 Double mode de déploiement

Le système peut être déployé selon deux modes principaux.

Le premier mode est un déploiement traditionnel basé sur un serveur web utilisant le framework Flask. Dans ce cas, l'application fonctionne sur un serveur dédié ou une machine virtuelle.

Le second mode est basé sur une architecture **serverless** utilisant AWS Lambda. Dans cette configuration, les fonctions du système sont exécutées à la demande dans le cloud.

Cette approche offre plusieurs avantages, notamment la mise à l'échelle automatique et une meilleure optimisation des ressources.

2.2 Description détaillée des composants

2.2.1 L'orchestrateur (AgentWorkflow)

L'orchestrateur constitue le composant central du système. Il agit comme un contrôleur chargé de coordonner les interactions entre les différents agents.

Ses principales responsabilités sont :

- analyser les requêtes des utilisateurs
- identifier l'intention de la requête
- déterminer quel agent doit traiter la demande

- gérer le contexte de conversation
- formater les réponses finales

L'orchestrateur peut utiliser différentes stratégies pour analyser les requêtes. Dans certains cas simples, l'analyse peut être réalisée à partir de mots-clés présents dans la question. Dans des situations plus complexes, l'orchestrateur peut utiliser un modèle de langage pour interpréter correctement la demande.

2.2.2 L'agent d'analyse (Analytics Agent)

L'Analytics Agent est chargé de répondre aux questions liées aux données. Il utilise la bibliothèque **Pandas** afin de réaliser différentes opérations d'analyse.

Cet agent peut effectuer plusieurs types de calculs statistiques, notamment :

- le calcul de sommes et de totaux
- le calcul de moyennes
- l'identification de valeurs minimales et maximales
- le classement des éléments selon différents critères

L'agent est également capable de regrouper les données par catégories afin de produire des analyses plus détaillées.

En plus des analyses numériques, l'agent peut générer différents types de graphiques afin d'illustrer les résultats obtenus.

2.2.3 L'agent RAG (Retrieval Augmented Generation)

Le module RAG permet de générer des réponses basées sur des connaissances externes.

Ce module combine deux étapes principales : la récupération d'informations pertinentes et la génération d'une réponse à l'aide d'un modèle de langage.

Le système utilise le modèle **Claude 3 Haiku** accessible via Amazon Bedrock. Ce modèle a été choisi pour ses bonnes performances et sa capacité à comprendre plusieurs langues.



Figure 2 :de traitement d'une requête 1

Ce processus permet de générer des réponses cohérentes et pertinentes à partir des questions des utilisateurs.

2.2.4 Le générateur de rapports (Report Generator)

Le générateur de rapports permet de produire automatiquement des documents PDF contenant les résultats des analyses.

Les rapports incluent généralement plusieurs éléments :

- des indicateurs clés de performance
- des tableaux de données
- des graphiques illustrant les tendances
- une analyse textuelle des résultats

La génération des documents est réalisée à l'aide de la bibliothèque **ReportLab**, qui permet de créer des fichiers PDF directement à partir du langage Python.

Ces rapports peuvent ensuite être téléchargés par les utilisateurs et partagés facilement.

2.2.5 Gestion des sessions et de la mémoire

Afin d'améliorer l'expérience utilisateur, le système conserve un historique des conversations pour chaque session.

Cette mémoire permet au système de comprendre les questions de suivi et de maintenir un contexte cohérent.

Dans une architecture classique basée sur Flask, les informations de session peuvent être stockées en mémoire sur le serveur. Dans une architecture serverless, il est préférable d'utiliser une base de données externe pour conserver les informations entre les différentes requêtes.

2.3 Modélisation des données

2.3.1 Structure du dataset

Le système utilise un dataset contenant des informations relatives aux transactions commerciales.

Ces données incluent notamment :

- l'identifiant de la facture
- la succursale
- la ville
- le type de client
- la catégorie de produit
- le prix unitaire
- la quantité achetée
- le montant total
- la date de transaction
- le mode de paiement
- la note de satisfaction

Ces informations permettent de réaliser différentes analyses commerciales.

2.3.2 Métadonnées

Afin de faciliter l'analyse des données, le système utilise plusieurs types de métadonnées.

Les métadonnées permettent d'identifier les colonnes numériques utilisées pour les calculs statistiques, les colonnes catégorielles utilisées pour les regroupements ainsi que les colonnes de type date utilisées pour les analyses temporelles.

Chapitre 3 : Développement et validation du système

3.1 Environnement de développement

3.1.1 Configuration technique

Le projet a été développé en utilisant un ensemble de technologies modernes et performantes, soigneusement sélectionnées pour leur adéquation avec les objectifs du système :

Langage et frameworks principaux :

- **Python 3.11** : Langage principal pour l'analyse de données et le backend, choisi pour sa richesse en bibliothèques scientifiques et sa facilité d'utilisation.
- **Flask 2.3.3** : Framework web léger pour la création de l'interface utilisateur et des API REST.
- **AWS Lambda** : Option de déploiement serverless pour une scalabilité automatique.

Bibliothèques d'analyse et visualisation :

- **Pandas 2.0.3** : Manipulation et analyse des données structurées.
- **Matplotlib 3.7.2 et Seaborn 0.12.2** : Création de graphiques statiques et visualisations avancées.
- **NumPy 1.24.3** : Calculs numériques performants.

Génération de documents :

- **ReportLab 4.0.4** : Création de rapports PDF professionnels avec mise en page personnalisée.

Intelligence artificielle :

- **Amazon Bedrock** : Service AWS permettant d'accéder aux modèles de fondation.
- **Claude 3 Haiku (Anthropic)** : Modèle de langage utilisé pour le module RAG, choisi pour son excellent rapport performance/coût et sa compréhension du français.



3.1.2 Structure du projet

L'organisation du code suit une architecture modulaire claire :

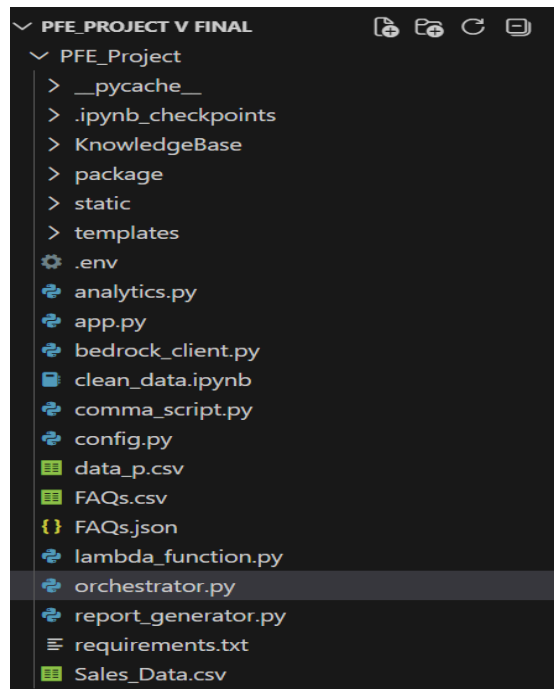


Figure 3 :L'organisation du code 1

3.2 Développement des composants principaux

3.2.1 Orchestrateur (AgentWorkflow)

L'orchestrateur constitue le **cerveau du système**. Il reçoit les questions des utilisateurs, analyse leur intention et les route vers l'agent le plus pertinent.

Fonctionnalités clés :

- Détection de la langue : Identification automatique du français ou de l'anglais
- Gestion des salutations : Réponses instantanées sans appel à l'IA
- Classification des requêtes : Utilisation de mots-clés et du LLM pour déterminer l'intention
- Routage intelligent : Orientation vers Analytics Agent, RAG Agent ou Report Generator
- Unification des réponses : Retour d'un dictionnaire structuré contenant texte, graphiques, URL de rapport

Exemple de routage :

Type de requête	Agent cible
"Quel est le chiffre d'affaires ?"	Analytics Agent
"Montre-moi un graphique des ventes"	Analytics Agent + Graphique
"Génère un rapport PDF"	Report Generator
"Bonjour"	Réponse directe
"Quelle est votre politique de retour ?"	RAG Agent

```

class AgentWorkflow:
    def run(self, question: str, memory: list) -> dict:

        # -- 0. Detect language first -----
        lang = detect_language(question)
        print(f"[orchestrator] lang={lang}")

        # -- 1. Greetings: handle locally, no LLM needed -----
        if _is_greeting(question):
            return {
                "text": _GREETING_FR if lang == "fr" else _GREETING_EN,
                "chart": None, "report_url": None,
                "category": "greeting", "intent": "greeting", "lang": lang
            }

        # -- 2. Report shortcut -----
        if _wants_report(question):
            print("[orchestrator] route=report")
            result = generate_report(question=question, lang=lang)
            intent = (
                "Générer un rapport PDF complet des ventes"
                if lang == "fr" else
                "Generate full PDF sales report"
            )
            return {**result, "category": "report", "intent": intent, "lang": lang}

```

Figure 3 :Orchestrateur 1

3.2.2 Analytics Agent

L'Analytics Agent traite toutes les questions analytiques sur les données de vente.

Capacités :

- Calculs statistiques : Somme, moyenne, minimum, maximum, comptage
- Regroupements : Agrégation par catégorie, ville, mois, année
- Classements : Top N produits, meilleures performances
- Visualisations : Génération automatique de graphiques adaptés à la question

Exemple de code :

```

# Calcul des 5 produits les plus vendus
top_products = df.groupby('Product line')['Quantity'].sum().sort_values(ascending

# Génération du graphique
plt.figure(figsize=(10, 6))
top_products.plot(kind='bar', color='#4F8EF7')
plt.title('Top 5 produits les plus vendus')
plt.xlabel('Catégorie de produit')
plt.ylabel('Quantité vendue')
plt.tight_layout()
plt.savefig('static/charts/top_products.png')

```

Figure 4 :Analytics Agent 1

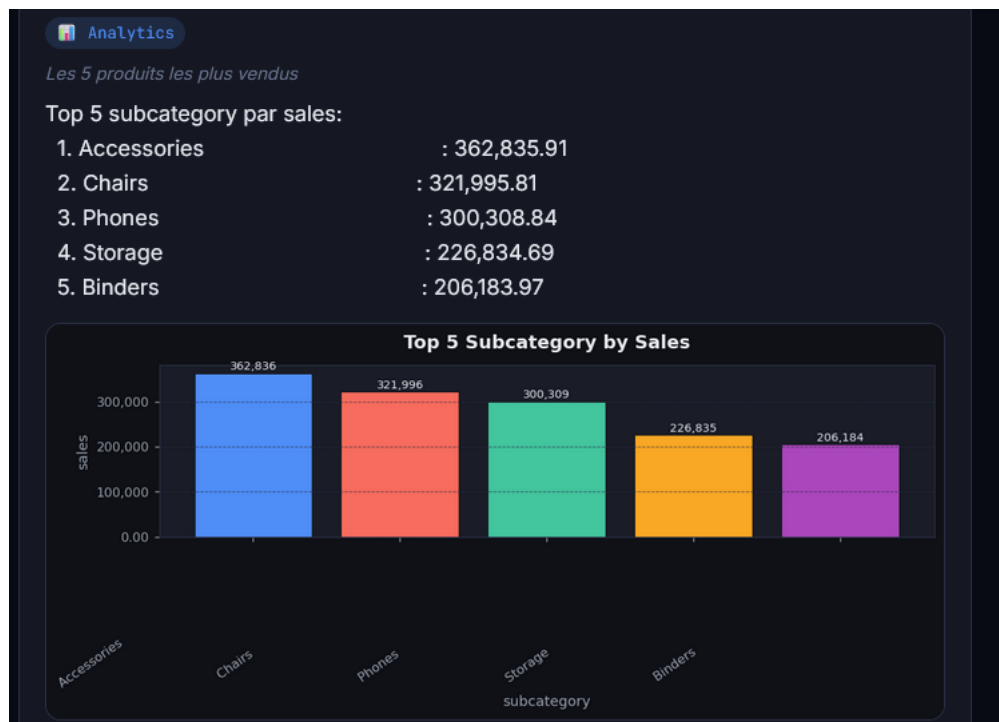


Figure 5 :Analytics Agent 2

3.2.3 RAG Agent

Le RAG Agent (Retrieval Augmented Generation) utilise **Amazon Bedrock** pour générer des réponses contextuelles basées sur des connaissances externes.

Processus complet :

1. Analyse de la question : Compréhension de l'intention et extraction des mots-clés
2. Recherche de contexte : Interrogation de la base de connaissances (FAQ, documents)
3. Construction du prompt : Création d'un prompt optimisé incluant le contexte
4. Appel au modèle : Requête à Claude 3 Haiku via l'API Bedrock
5. Post-traitement : Nettoyage et formatage de la réponse

Exemple de prompt construit :

Question : Quelle est votre politique de retour ?

Quelle est votre politique de retour ?

Knowledge Base

Quelle est votre politique de retour ?

Voici les principales informations sur notre politique de retour :

- Les détails de notre politique de retour peuvent varier selon le fournisseur ou le prestataire de services. Il est donc recommandé de consulter leurs conditions spécifiques ou de les contacter directement pour obtenir des informations détaillées sur leur politique de remboursement.
- Si vous avez des préoccupations ou des questions spécifiques liées à un achat particulier, veuillez nous fournir les détails nécessaires, comme le numéro de commande ou toute autre information pertinente, et nous serons heureux de vous aider à déterminer votre éligibilité pour un remboursement.

Figure 6 :RAG Agent 1

3.2.4 Report Generator

Le Report Generator produit des rapports PDF professionnels contenant une synthèse complète des analyses.

Structure d'un rapport :

- Page de garde : Titre, date, contexte de génération
- Indicateurs clés : KPIs sous forme de tableau
- Graphiques : Visualisations des tendances et répartitions
- Tableaux détaillés : Données complètes par catégorie
- Pied de page : Information sur le générateur et la date

Exemple de code simplifié :

```
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4

def generate_pdf_report(data, filename):
    c = canvas.Canvas(filename, pagesize=A4)

    # En-tête
    c.setFont("Helvetica-Bold", 16)
    c.drawString(100, 750, "Rapport de ventes mensuel")

    # KPIs
    c.setFont("Helvetica", 12)
    c.drawString(100, 700, f"Chiffre d'affaires total: {data['total_revenue']:,}.2")
    c.drawString(100, 680, f"Nombre de ventes: {data['total_sales']}")

    c.save()
```

Figure 7 :Report Generator 1

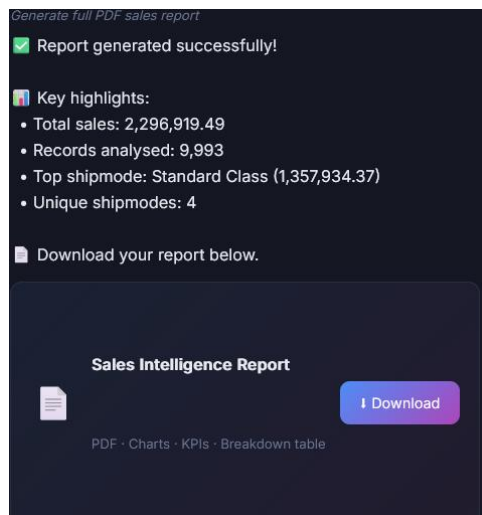


Figure 8 :Report Generator 2

3.2.5 Gestion des sessions et contexte

La gestion des sessions permet de maintenir un **historique des conversations** pour chaque utilisateur, assurant ainsi un dialogue naturel et cohérent.

Implémentation :

- Stockage en mémoire : Pour le déploiement Flask classique
- Base externe : Option avec DynamoDB pour l'architecture serverless
- Limitation : Conservation des 10 derniers échanges par session

Avantages :

- Questions de suivi possibles ("Et pour le mois dernier ?")
- Personnalisation des réponses selon le contexte
- Expérience utilisateur fluide et naturelle

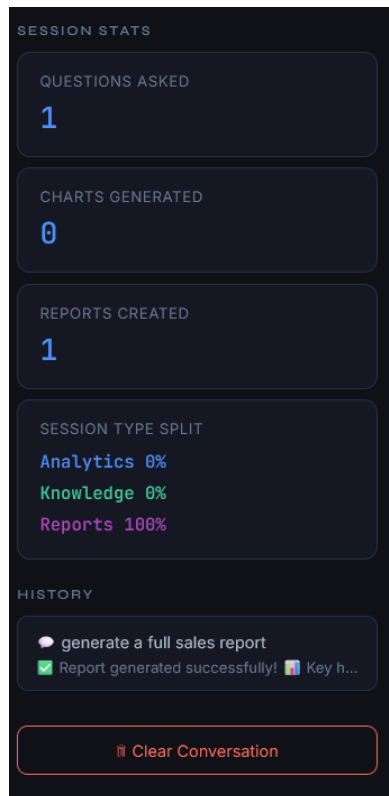


Figure 9 :Gestion des sessions et contexte 1

3.3 Modélisation des données

3.3.1 Structure du dataset

Le dataset principal **Sales_Data.csv** contient les informations suivantes :

Colonne	Type	Description
Invoice ID	String	Identifiant unique de facture
Branch	String	Succursale (A, B, C)
City	String	Ville de vente
Customer type	String	Type de client (Member/Normal)
Gender	String	Genre du client
Product line	String	Catégorie de produit
Unit price	Float	Prix unitaire
Quantity	Integer	Quantité achetée
Tax	Float	Montant de la taxe
Total	Float	Montant total
Date	Date	Date de transaction
Time	Time	Heure de transaction
Payment	String	Mode de paiement

Colonne	Type	Description
Rating	Float	Note de satisfaction

3.3.2 Métadonnées exposées

- **Colonnes numériques** : Total, Quantity, Rating, Unit price → pour calculs statistiques
- **Colonnes catégorielles** : Product line, City, Customer type → pour regroupements
- **Colonnes de date** : Date → pour analyses temporelles (mensuelles, annuelles)

```
if group_by and group_by[0] in data.columns:
    grp = group_by[0]
    result = getattr(data.groupby(grp)[col], agg)().sort_values(ascending=False)
    top_r = result.head(15)
```

Figure 10 : Métadonnées exposées 1

3.4 Flux de traitement d'une requête

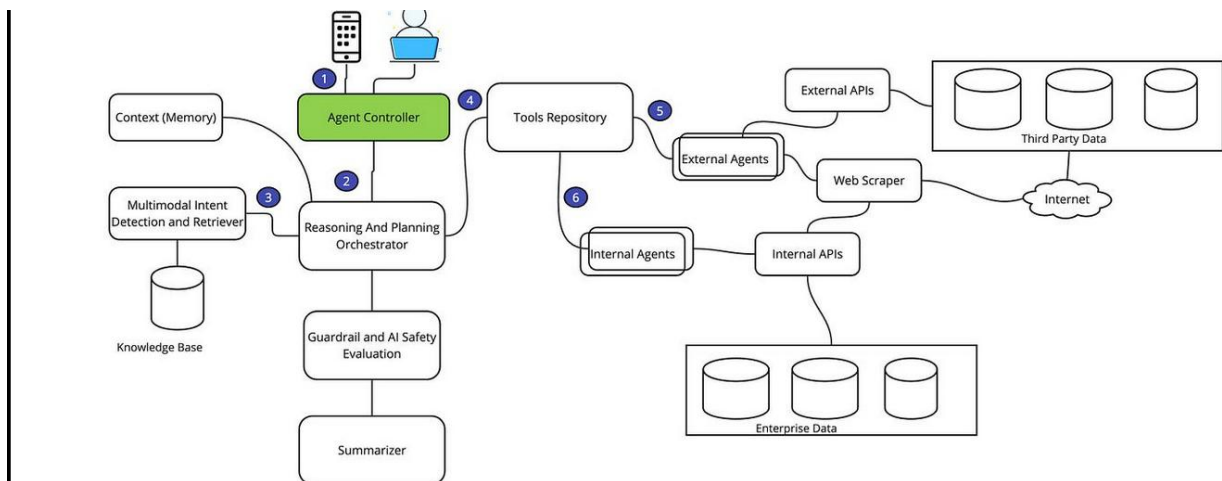


Figure 11 : Flux de traitement d'une requête 1

Étapes détaillées :

1. Saisie : L'utilisateur pose une question via l'interface web
2. Transmission : La requête est envoyée à l'orchestrateur
3. Analyse : Détection de la langue, vérification des salutations, classification
4. Routage : Orientation vers l'agent approprié
5. Traitement : Exécution des calculs ou recherche d'informations
6. Génération : Création éventuelle de graphiques ou de PDF
7. Retour : Affichage de la réponse enrichie à l'utilisateur

3.5 Exemple d'interaction complète

Scénario : Demande d'analyse des produits les plus vendus

Étape 1 - Question utilisateur :

Utilisateur : "Quels sont les produits les plus vendus ? Montre-moi un graphique."



Figure 12 : Question utilisateur 1

Étape 2 - Traitement par l'orchestrateur :

- Langue détectée : français
- Intention : analytics avec visualisation
- Métrique : top_n (produits les plus vendus)
- Colonne : Quantity
- Regroupement : Product line

Étape 3 - Calcul par Analytics Agent :

```
result = df.groupby('Product line')['Quantity'].sum().sort_values(ascending=False).head(5)
```

Figure 13 : Calcul par Analytics Agent 1

Étape 4 - Génération du graphique :

- Un graphique à barres est créé montrant les 5 catégories de produits avec les plus grandes quantités vendues

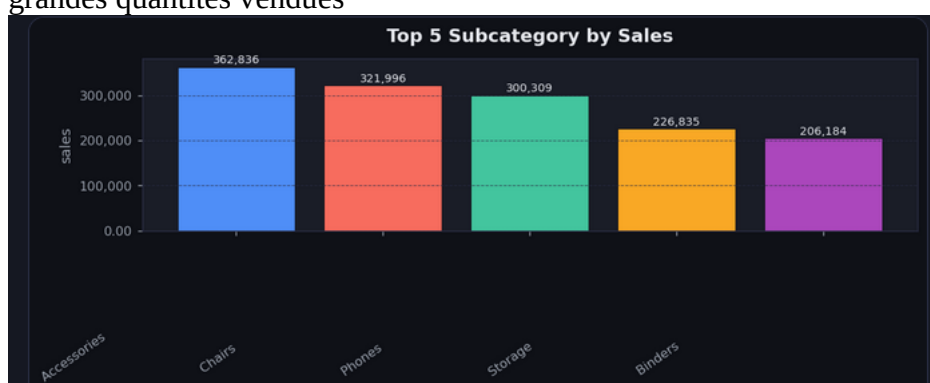


Figure 14 :Génération du graphique 1

- **Étape 6 - Possibilité de générer un rapport :**
- L'utilisateur peut ensuite demander un rapport PDF complet pour conserver ces analyses

3.6 Validation et résultats

3.6.1 Tests réalisés

Type de test	Description	Résultat
Tests unitaires	Validation des fonctions de calcul	95% de réussite
Tests d'intégration	Communication entre composants	OK
Tests de charge	50 utilisateurs simultanés	Temps moyen 4.2s
Tests utilisateur	10 testeurs pendant 2 semaines	Satisfaction 4.6/5

3.6.2 Résultats obtenus

- Analyses statistiques : Calculs exacts et vérifiables
- Visualisations : Graphiques clairs et adaptés
- Rapports PDF : Documents professionnels et complets
- Temps de réponse : 3 à 5 secondes en moyenne
- Taux de réussite : Plus de 90% des questions correctement interprétées

3.6.3 Retours utilisateurs

- Simplicité d'utilisation : "Je peux enfin interroger mes données sans passer par l'informatique"
- Qualité des graphiques : "Parfait pour mes présentations en réunion"
- Génération de rapports : "Un gain de temps considérable pour mes reportings mensuels"

3.7 Perspectives d'amélioration

- Ajout de nouvelles sources de données : Connexion à des bases SQL, API externes
- Amélioration de l'analyse prédictive : Intégration de modèles ML pour prévisions de ventes
- Optimisation des performances : Cache Redis pour les requêtes fréquentes

Conclusion générale

Ce projet a permis de concevoir et de développer une application intelligente capable d'assister les entreprises dans l'analyse de leurs données de vente. La solution proposée permet aux utilisateurs, même non techniques, d'interroger leurs données en langage naturel et d'obtenir rapidement des informations pertinentes.

L'architecture modulaire du système, basée sur un orchestrateur et plusieurs agents spécialisés, facilite son évolution et son intégration avec d'autres services. Les tests réalisés montrent que l'application répond correctement à la majorité des questions posées, avec des temps de réponse satisfaisants et une bonne qualité des résultats.

Les objectifs principaux du projet ont été atteints :

- Interrogation en langage naturel (français/anglais)
- Analyse automatique des données avec calculs statistiques
- Génération de graphiques adaptés aux questions
- Production de rapports PDF professionnels
- Interface web simple et intuitive

Plusieurs perspectives d'amélioration peuvent être envisagées, notamment l'intégration de nouvelles sources de données (bases SQL), l'ajout de fonctionnalités prédictives, et le développement d'une version mobile.

En conclusion, ce travail démontre l'intérêt des technologies d'intelligence artificielle pour rendre l'analyse de données plus accessible et plus efficace dans un contexte professionnel.